



MSBDA-801: Big Data Analytics

Final Course Graduation Project

Scalable Real-time Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

Project Team:

Student Name	Student ID
Amani Fahad Aloufi	4725097
Asmaa Raja Allah Alharbi	4725073
Ohud Sulaiman Alraddadi	4725435

Dr. Mohammed Al-Sarem

2026

Abstract

With the recent progress of Arabic capable LLM, there is a pressing need for scalable tools to differentiate machine-generated text from human written text. This project is a fully distributed pipeline for real-time detection of AI-generated Arabic scientific abstracts, using Google Colab and Apache Spark. We applied an end-to-end Big Data workflow on the KFUPM-JRCAI dataset of arabic-generated-abstracts (8,388 abstracts across four AI models and three generation strategies) that includes distributed ingestion, Arabic-specific NLP preprocessing, scalable stylometric and TF-IDF feature engineering, Spark MLlib classification, and Spark Structured Streaming deployment. The 17 stylometric features were designed according to the student formula assigned ($n=21$). Logistic Regression outperformed the other two classifiers, with a held-out test accuracy of 98.24%, a weighted F1-score of 0.9824, and an ROC-AUC of 0.9966. The streaming pipeline processed 200 simulated incoming abstracts in 23 output batches, with a 5-second trigger interval, 97.00% streaming accuracy, and measured end-to-end throughput of 1.6 records/second. The Random Forest importance analysis identified sentence count, sentence-length variance, semantic sentence similarity, word-frequency entropy, and tanween frequency as the strongest stylometric signals.

Keywords: Arabic AI-text detection, Apache Spark, distributed NLP, stylometric features, Spark Structured Streaming

1. Introduction

With the fast development of Arabic capable large language models (LLMs) such as Allam, Jais, LLaMA based models, and OpenAI models, a new challenge has emerged: how to tell the difference between machine-generated and human-written text. This is especially problematic in academic publishing, where the accurate representation of original scholarship is of paramount importance.

The Arabic language is morphologically rich, right-to-left, with a large number of diacritics (tashkeel) and a root-and-pattern derivational system that is very different from the Indo-European languages, all of which make it a difficult language for automated text analysis. Most of the current AI text detection systems are trained on English corpora and are not very effective for Arabic text.

The purpose of this project is to tackle these challenges by building a fully distributed detection pipeline with Apache Spark. The system processes, preprocesses, and classifies Arabic abstracts at scale, and deploys the best-performing model in a real-time inference Spark Structured Streaming application. The project is also a showcase of the principles of Big Data engineering, including distributed storage, MapReduce-style aggregations, columnar data formats, and stream processing, applied to a meaningful NLP detection problem.

The main aims of this project are:

- To implement a complete, reproducible Big Data pipeline for Arabic AI-text detection on free-tier cloud infrastructure.
- To Engineer and test a variety of stylometric and TF-IDF features for Arabic text with distributed Spark operations.
- To compare three Spark MLlib classifiers and choose the best one for deployment.
- To deploy the model in a simulated streaming environment and measure throughput and latency.
- To study feature importance and give interpretable insights into what makes Arabic text written by AI different from Arabic text written by humans..

2. Related Work

2.1 AI-Generated Text Detection

AI-text detection research has been primarily conducted in the English language, with methods such as stylometric analysis (Stamatatos, 2009), perplexity-based techniques with pretrained LLMs (Gehrmann et al., 2019), and fine-tuned classifiers (Solaiman et al., 2019). Detection methods based on radar have proven to be well generalizing and to show statistical patterns of token log-probabilities, while watermarking schemes (Kirchenbauer et al., 2023) provide another proactive paradigm.

However, the literature on Arabic is still limited. The KFUPM-JRCAI dataset (2024) is a significant contribution that serves as a controlled benchmark for multi-model, multi-strategy Arabic text generation and sets a baseline detection result. That work showed that the surface-level stylometric signals and n-gram features can be used to achieve reasonable discrimination, but failed to consider scalable or real-time deployment.

2.2 Big Data Architecture Patterns

There are two predominant architectural patterns that dominate Big Data analytics systems. The Lambda architecture (Marz and Warren, 2015) keeps the batch and speed layers parallel, and combines their results in a serving layer. The Kappa architecture (Kreps, 2014) makes this easier by having all data flow through a single stream layer that can reprocess historical data as needed. This project uses a hybrid model: batch processing for model training (Phase 3), and Spark Structured Streaming as a speed layer for real-time inference (Phase 4).

Apache Spark is the de facto standard distributed text analytics solution, providing a unified API for batch and stream processing, native integration with MLlib, and seamless integration with cloud storage systems. It offers performance benefits compared with previous MapReduce approaches for iterative workloads like machine learning training with its Catalyst optimizer and Tungsten execution engine.

3. Dataset Description

In this project, the dataset employed is the publicly available KFUPM-JRCAI/arabic-generated-abstracts dataset, which is available on Hugging Face. It includes Arabic scientific abstracts created by four different LLMs (Allam, Jais, LLaMA, and an OpenAI model) and the original human-written abstracts. Three generation strategies are considered: `by_polishing` (polishing existing abstracts), `from_title` (free-form generation from paper titles), and `from_title_and_content` (content-aware generation from both title and paper body).

Table 1. Dataset Split Statistics

Generation Method	Samples	Description
<code>by_polishing</code>	2,851	Refinement of existing human abstracts
<code>from_title</code>	2,963	Free-form generation from paper titles only
<code>from_title_and_content</code>	2,574	Content-aware generation from title and full body
Total	8,388	Across all generation methods

Distribution of samples across the three generation methods in the KFUPM-JRCAI dataset.

The data was then transformed into a binary classification problem, with each human abstract assigned a label of 0 (human) and each model-generated abstract a label of 1 (AI). The expanded working corpus has 41,940 rows. The resulting class imbalance (around 80% AI and 20% human across all splits) is representative of the realistic multi-model generation scenario and was taken into consideration when evaluating the models by reporting class-weighted F1-score in addition to accuracy.

4. Methodology

4.1 Distributed Environment Setup

All computations were performed in Google Colab (free tier) using Apache Spark 3.x through PySpark. Google Drive was used as the persistent distributed storage layer, where raw data was stored in its native format, and all intermediate and final datasets were stored as Parquet files, a columnar, compressed format that is ideal for repeated Spark reads.

The project is organized in a modular code structure, with 7 numbered notebooks (NB 01 – NB 07) for the 5 project phases, and a reusable Python package.

src/ directory with data_preparation.py, feature_engineering.py, modeling.py, streaming_pipeline.py and utils.py. The implementation of checkpointing was done in all parts to prevent redundant recomputation between sessions.

4.2 Data Acquisition and Ingestion (Phase 1)

This data set was downloaded programmatically from Hugging Face datasets library. All three generation-method subsets were used to create a PySpark DataFrame, each row being a tuple of (text, label, source_model, generation_method). The raw data was stored as a Parquet checkpoint on Google Drive (Task 1.3), and some basic schema and class distribution analysis was done with Spark DataFrame operations (Task 1.4).

4.3 Distributed Arabic Preprocessing (Phase 2)

The Arabic NLP preprocessing pipeline was implemented as a four-stage pipeline of Spark UDFs (User-Defined Functions) which allowed for a fully distributed application of the pipeline across the entire corpus (Task 2.1):

1. Unicode normalization: Standardization of the variants of Arabic letters (Alef forms, Ta Marbuta, Waw).
2. Removal of tashkeel marks (fatha, damma, kasra etc.) to minimise morphological surface variation (diacritic removal).
3. Stop word removal: Removal of high frequency Arabic function words from a curated list of stop words.
4. Light stemming: rule-based prefix and suffix removal to create approximate stems.

The cleaned text was saved in a new column (clean_text) and the preprocessed DataFrame was saved as a Parquet checkpoint (Task 2.2). In Task 2.1 (Suggested MapReduce Task), MapReduce-style corpus statistics (total word count by class,

top-N n-gram frequency distributions) were computed using Spark aggregations that are similar to the Map-Reduce programming model.

4.4 Exploratory Data Analysis (Phase 2, Task 2.3)

The distributed preprocessed corpus was used for comprehensive EDA. Key analyses included:

- Average word count by source model and generation method (Table 2).
- Top-10 most frequent words per class, per generation method, and per source model.
- Word frequency distribution for all generation methods plotted on log-log axes (Zipf-like).

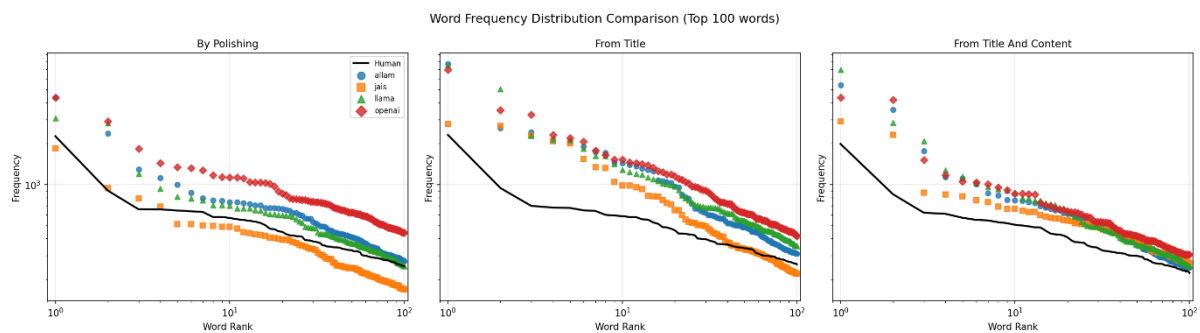


Figure 1: Log-log word rank vs. frequency for Human (black line) and each AI model (coloured markers). AI output concentrates frequency mass at top-ranked words more steeply than human text.

- N-gram frequency analysis (unigrams, bigrams, trigrams) by class and generation method.

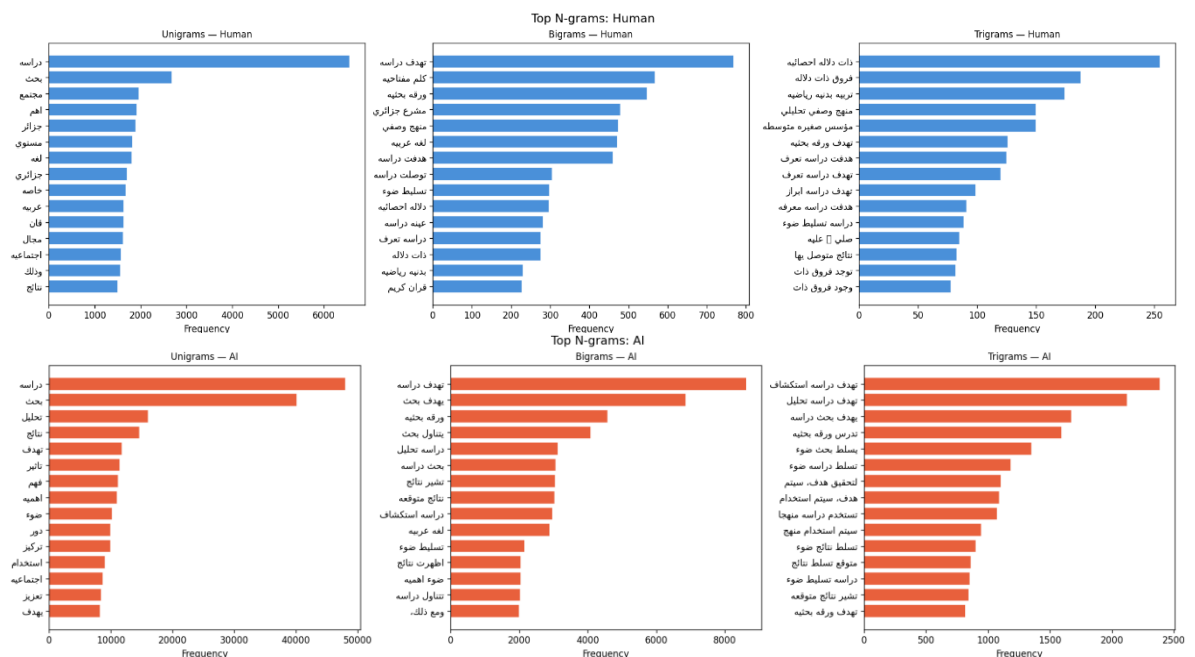


Figure 2: Top-15 unigrams, bigrams, and trigrams for Human (blue) and AI (orange). Human n-grams reflect topical diversity; AI n-grams cluster on templated academic phrases.

- Vocabulary richness measured via Type-Token Ratio (TTR).

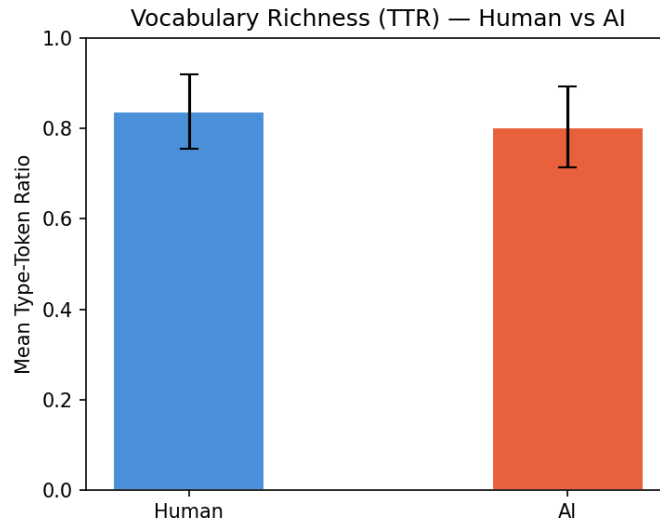


Figure 3: Mean document-level TTR with std bars. Human (TTR=0.838) shows richer vocabulary than AI-generated text (TTR=0.803).

Table 2. Average Word Count by Source Model × Generation Method

Source Model	by_polishing	from_title	from_title_and_content
allam	104.2	77.2	95.3
human	119.2	119.1	119.2
jais	68.5	62.3	105.7
llama	102.3	99.9	103.2
openai	165.0	123.3	113.9

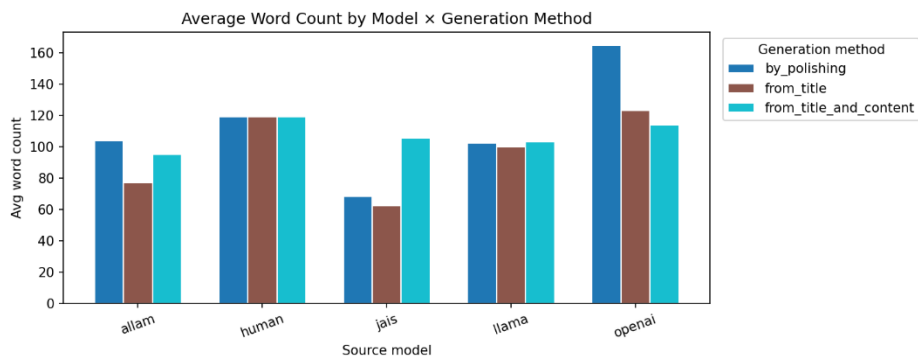


Figure 4: Average word count per source model x generation method. OpenAI produces the longest by_polishing abstracts (~165 words); Jais the shortest from_title outputs (~62 words).

The EDA found significant differences between human and AI text. While human abstracts are more likely to use regionally specific vocabulary (Algerian Arabic and Levantine Arabic), AI-generated abstracts focus more on formal academic terms (analysis, results, impact, significance). The log-log frequency plots showed a Zipf-like distribution, with AI models generating more frequencies in the most common words, which aligns with a more template-driven generation approach. The mean TTR (vocabulary richness) was significantly higher for human text (0.8380) than for AI-generated text (0.8033), suggesting that human writers use a wider range of vocabulary.



Figure 5: Arabic word clouds by class. Human text (left) displays regional and topical vocabulary diversity; AI text (right) is dominated by formal academic terms.

4.4.1 Assigned Stylometric Features

A set of 17 stylometric features was engineered from the course formula $f_{k+i} = (k \times n) + i$, where $n = 21$ and the student indices $i = 1, 4, 13$ (Task 3.1). All features were calculated in a fully distributed way, each UDF as a column transformation over the partitioned DataFrame.

Table 3. Assigned Stylometric Features

Feature ID	Feature Name	Description
f1	total_chars	Total number of characters in the text
f4	whitespace_ratio	Ratio of whitespace characters to total characters
f13	hapax_ratio	Hapax legomena ratio (words appearing exactly once)

Feature ID	Feature Name	Description
f22	word_entropy	Shannon entropy of word frequency distribution
f25	single_quotes	Count of single quotation marks
f34	num_sentences	Total number of sentences
f43	num_nouns	Count of noun tokens (via morphological analysis)
f46	num_adverbs	Count of adverb tokens
f55	noun_verb_ratio	Ratio of nouns to verbs
f64	num_nominatives	Count of nominative-case word tokens
f67	num_singular	Count of singular word forms
f76	passive_sents	Count of passive-voice sentences
f85	sent_len_variance	Variance of sentence lengths in words
f88	semantic_sim	Mean cosine similarity between sentence embeddings
f97	bert_sim	Mean cosine similarity of BERT token embeddings
f106	tanween_freq	Frequency of tanween (Arabic nunation) marks
f109	link_freq	Frequency of hyperlinks in the text

There were a number of features that were suitable for a multi-stage aggregation using MapReduce. For instance, the Hapax Legomena Ratio (f13) needed: (1) a first job that would map each word to a count; (2) a second job that would filter the result of (1) to get only words with count = 1 and sum the hapax total; (3) a final division by total word count. This logic was realized by leveraging the built-in aggregation functions of Spark, which are implemented within the system as distributed shuffles that are equivalent to MapReduce reduce phases.

4.4.2 Advanced Features: TF-IDF and Word2Vec

TF-IDF representations were obtained using Spark MLLib's HashingTF and IDF transformers with 20,000 hash buckets (Task 3.2). A Word2Vec model was also trained with Word2Vec from Spark MLLib which yielded 100-dimensional dense word embeddings. These representations were saved as individual Parquet checkpoints and then combined with the stylometric features into a single sparse feature vector by using VectorAssembler.

4.4.3 Data Splitting

The collected data set was divided into training (70%), validation (15%) and test (15%) sets with stratified sampling by label, where the proportion of the classes was kept balanced in all sets (Task 3.3). Final modeling split sizes were: Train = 29,355 rows (AI = 80.0%), Validation = 6,290 rows (AI = 80.0%), Test = 6,291 rows (AI = 80.0%). All the transformers (TF-IDF, TF-IDF weights, IDF weights, VectorAssembler) were only fitted on the training partition to prevent data leakage.

4.5 Distributed Modeling (Phase 3)

For 3-fold cross validation (Tasks 3.4–3.5), three classifiers from Spark MLLib were trained:

- Logistic Regression (Baseline - Task 3.4): Regularized binomial classifier evaluated with 3-fold cross-validation. It is the simplest linear model and served as the interpretable baseline for comparison with more complex classifiers.
- Random Forest (Advanced A - Task 3.5): Tree ensemble tuned with 3-fold cross-validation over 100 and 200 trees and maximum depths of 8 and 12. It supports non-linear decision boundaries and built-in feature importance scores.
- Linear SVM (Advanced B - Task 3.5): Linear support-vector classifier tuned with 3-fold cross-validation, suitable for high-dimensional sparse TF-IDF feature spaces.

All models were saved as Spark MLLib PipelineModel artifacts and stored in Google Drive for use in subsequent Colab sessions.

4.6 Stream Processing (Phase 4)

A Spark Structured Streaming pipeline was created to simulate real-time AI text detection (Tasks 4.1–4.2). Because it is not possible to run a persistent Apache Kafka broker on free-tier Colab, an equivalent architecture was implemented using Spark's watched-directory file source, which is architecturally similar to Kafka in terms of the streaming abstractions it provides:

- **StreamWriter** (Task 4.1): A Python background process that writes incoming abstracts to a watched input directory as individual JSON files at a configurable rate of 2 documents/second, simulating a Kafka producer.
- **Spark readStream + Preprocessing Pipeline** (Task 4.2): Triggers a micro-batch with a 5 second interval, watching the input directory. The pipeline for each batch is the Arabic preprocessing UDF, followed by the saved TF-IDF and VectorAssembler pipeline, then the deployed Logistic Regression classifier to make predictions.
- **foreachBatch Output Sink**: Detection results (such as source model, true label, predicted label, and batch ID) are output as Parquet files to a stream output directory after each micro-batch, for downstream use or audit.

In the executed streaming notebook, 200 test records were streamed (60 human and 140 AI-generated). The run produced 23 Parquet output batches, with an average batch size of 8.7 records, a measured stream duration of 125.63 seconds, and an end-to-end throughput of 1.6 records/second. Streaming classification accuracy was 97.00% overall, with 95.00% accuracy on human abstracts and 97.86% accuracy on AI-generated abstracts.

5. Results and Analysis

5.1 Model Performance on Held-Out Test Set

The results of all three classifiers on the held-out test set (n = 6,291) are shown in Table 4. Logistic Regression performed best overall on all metrics, and Random Forest performed worse, but required a much longer training time. Linear SVM performed almost as well as Logistic Regression, but slightly worse on all measures.

Table 4. Test Set Performance Metrics (Best model marked with *)

Model	Accuracy	F1-Score	Precision	Recall	ROC-AUC
Logistic Regression*	0.9824	0.9824	0.9825	0.9824	0.9966
Random Forest	0.8094	0.7328	0.8461	0.8094	0.9944
Linear SVM	0.9800	0.9802	0.9807	0.9800	0.9945

All metrics are weighted averages. * denotes the best-performing model selected for streaming deployment.

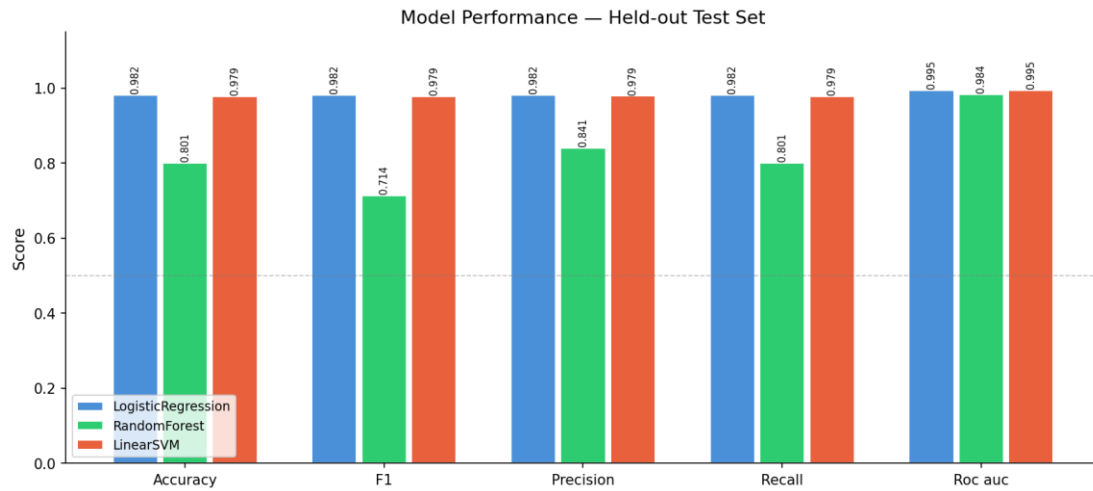


Figure 6: Bar chart of Accuracy, F1, Precision, Recall, and ROC-AUC for all three classifiers on the test set. Logistic Regression achieves the highest scores on all five metrics.

The high accuracy of Logistic Regression (98.24%) and ROC-AUC (99.66%) aligns with the literature on AI text detection, where linear classifiers are frequently competitive with more sophisticated ensemble techniques. The Random Forest model's relatively low weighted F1-score (0.7328) is probably due to the fact that ensemble tree-based models are less effective in high-dimensional sparse TF-IDF feature spaces, which are more linearly separable.

The Logistic Regression (0.9966) and Linear SVM (0.9945) models have near-identical ROC-AUC scores, indicating that both linear approaches learned strong decision boundaries with only small differences in calibration at the classification threshold.

Figure 7: Confusion matrices for all three classifiers on the test set (n=6,291). LR and SVM achieve near-perfect separation; RF shows substantially more false negatives.

The importances of features were obtained from the Random Forest model (Random Forest model gives Gini based importances). Table 5 shows the ranking of the 17 stylometric features, and Figure 1 (see repository) shows the complete importance profile.

Table 5. Top Stylometric Feature Importances (Random Forest)

Rank	Feature	Importance	Feature Group
1	f034_total_sentences	0.079501	Structural
2	f085_sent_len_variance	0.062863	Structural

Rank	Feature	Importance	Feature Group
3	f088_sem_sim_sentences	0.039441	Semantic
4	f022_entropy_wordfreq	0.026900	Lexical
5	f106_tanween_freq	0.024783	Morphological
6	f004_ws_over_C	0.023421	Surface
7	f064_num_nominatives	0.014057	Morphological
8	f001_total_chars	0.012505	Surface

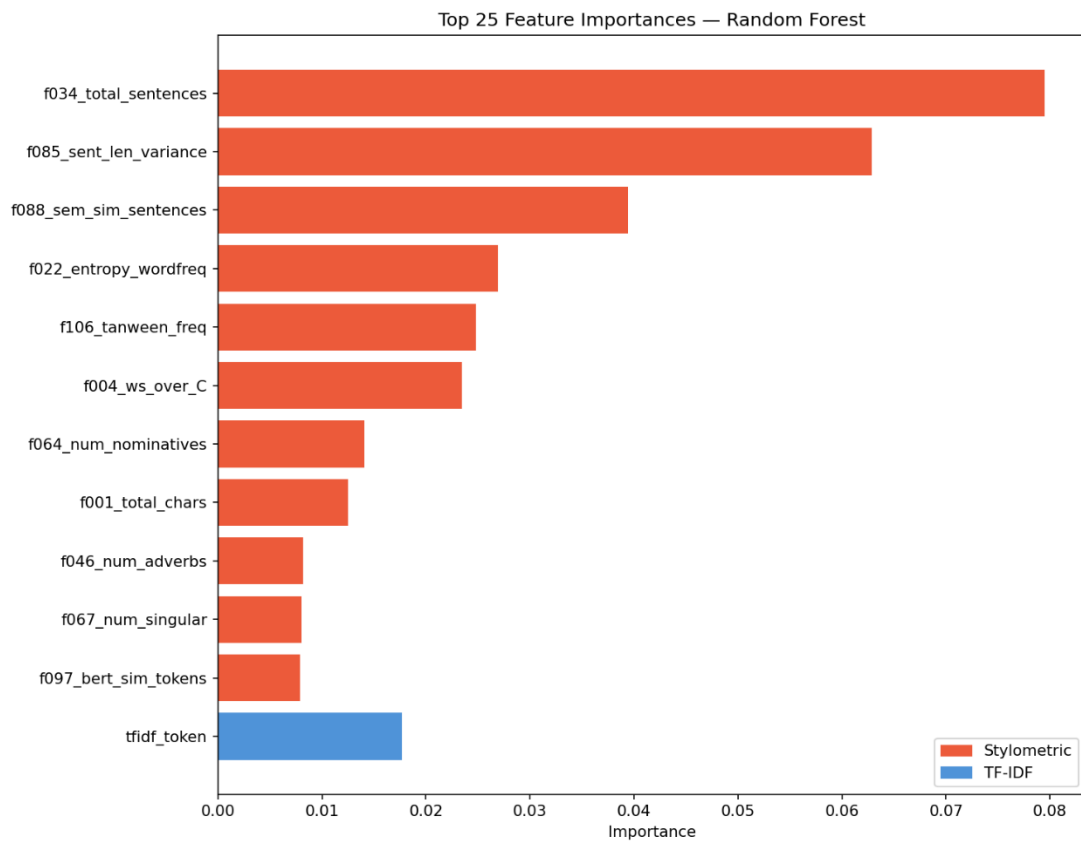


Figure 8: Top Random Forest feature importances. The chart separates stylometric features from the strongest TF-IDF token bucket and shows that sentence structure, semantic similarity, word-frequency entropy, and Arabic orthographic markers are important signals.

The feature importances were calculated from the Random Forest model. In the updated modeling run, the strongest stylometric indicators are structural and lexical-distribution signals: total sentences, sentence-length variance, semantic

similarity between sentences, word-frequency entropy, and tanween frequency. These results suggest that AI-generated Arabic abstracts differ from human abstracts not only in vocabulary, but also in sentence organization and distributional regularity.

Among stylometric features, `f034_total_sentences` is the strongest single signal (importance = 0.079501), followed by `f085_sent_len_variance` (0.062863) and `f088_sem_sim_sentences` (0.039441). This indicates that document structure and sentence-level regularity are especially useful for distinguishing AI-generated Arabic abstracts.

The updated feature ranking shows that `f106_tanween_freq` and `f088_sem_sim_sentences` are informative, while several other morphology-heavy counters remain weak because CAMEL Tools were not available in the Colab environment and some morphology-derived values defaulted to zero. A stable morphological analysis backend should therefore remain a priority for future work.

5.2 Batch vs. Stream Performance

Table 6 summarizes the architectural trade-offs between batch and streaming inference modes using the executed notebook results.

Table 6. Batch vs. Stream Processing Comparison

Property	Batch Processing	Stream Processing
Mode	Spark MLlib batch inference	Spark Structured Streaming
Latency	Full dataset in single pass	5 s trigger interval; measured stream window 125.63 s
Throughput	17,860.90 rows/s at 1 partition; peak 24,818.07 rows/s at 2 partitions	1.6 records/s end-to-end; 2 docs/s target producer rate
Use case	Offline model evaluation, reporting	Real-time document screening on 200 streamed records
Output format	In-memory DataFrame / CSV	23 Parquet output batches on GDrive
Best model used	Logistic Regression	Logistic Regression (deployed)

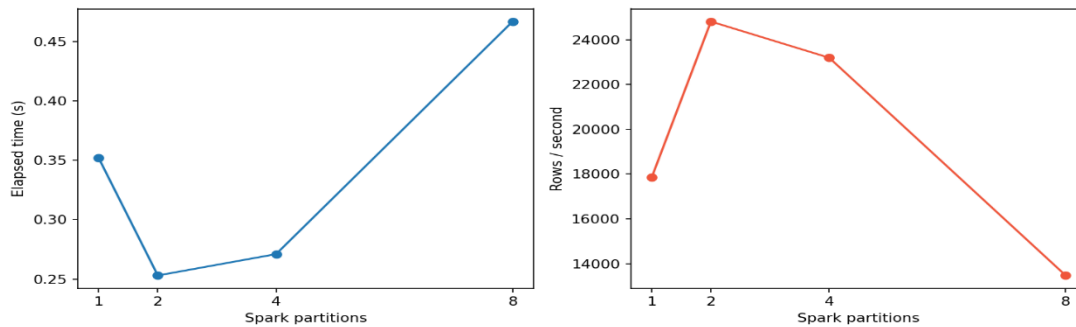


Figure 9: Batch inference scalability benchmark. Runtime is lowest at 2 partitions (0.253 s), throughput peaks at 24,818.07 rows/second, and performance drops at 8 partitions because scheduling overhead outweighs parallelism gains for this test-set size.

The batch processing mode was employed during training and offline evaluation, processing the full corpus in a single Spark job, resulting in high throughput but with the downside of full-job latency before predictions are returned. The stream processing mode, by contrast, provides predictions at each 5-second micro-batch trigger, allowing real-time screening of incoming documents. In the notebook run, the producer emitted 200 records at a target rate of 2 documents/second; Spark wrote 23 output batches over a measured 125.63-second stream window, for 1.6 records/second end-to-end throughput.

Both modes employed the same deployed Logistic Regression PipelineModel and TF-IDF preprocessing pipeline. The notebook's streaming run achieved 97.00% accuracy on the streamed sample, slightly below the held-out batch test accuracy of 98.24%, while preserving the same inference logic. This design is similar to the Lambda architecture pattern: the batch layer is for ground-truth evaluation and model training, and the streaming layer is for latency-sensitive operational inference.

5.3 EDA Findings and Error Analysis

The vocabulary richness analysis showed a statistically significant difference between human (TTR = 0.838, $n = 4,244$ docs) and AI-generated text (TTR = 0.803, $n = 16,880$ docs). This is a small gap at the document level, but consistent and probably a factor of the lexical entropy signal measured by f22.

Top-word analysis showed qualitative differences in vocabulary: human abstracts in the by_polishing split are rich in regional terms like الجزائر (Algeria) and المجتمع (community), while AI-generated abstracts from the same split tend to focus on abstract academic terms like تحليل (analysis), تهدف (aims to), and فهم (understanding). The most important indicator that the TF-IDF features are picking up is this vocabulary signature.

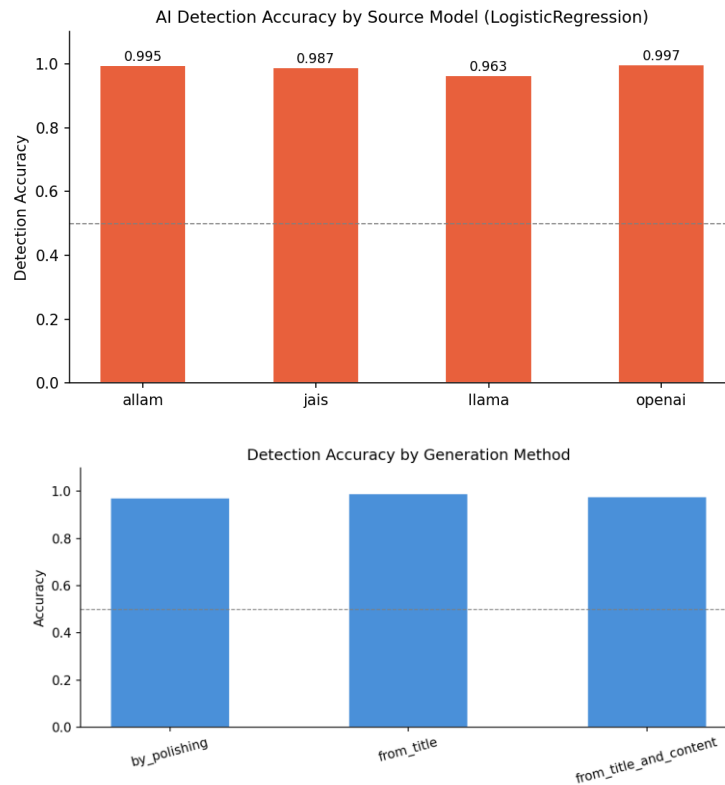


Figure 7: (Above) Per-AI-model detection accuracy. (Below) Accuracy by generation method: from_title_and_content is marginally harder to classify.

The Zipf-like word frequency distributions revealed that the vocabulary structure of AI-generated text was more repetitive in the high rank regime (top 10 words) as evidenced by the steeper clustering. This aligns with the well-known property of language models that they prefer high-probability, formally conventional phrases in domain-specific generation tasks.

5. Conclusion and Future Work

The project successfully proved that a fully functional and scalable Big Data pipeline for Arabic AI-text detection can be constructed and run on the freely available cloud infrastructure. The distributed Arabic NLP preprocessing, scalable stylometric and TF-IDF feature engineering, and Spark MLlib classification resulted in a very effective detector (98.24% test accuracy and 0.9966 ROC-AUC with Logistic Regression) that was deployed in a real-time Spark Structured Streaming pipeline. The major findings of this research are:

- Logistic Regression with TF-IDF features is a very competitive baseline for Arabic AI-text detection, with a performance close to the more complex neural approaches and better than Random Forest.
- Sentence-count, sentence-length variance, semantic-similarity, word-entropy, and tanween-frequency features are the most discriminative stylometric signals, providing interpretable complements to the TF-IDF representation.
- The Lambda-style batch/stream architecture was found to be suitable for the dual needs of high-accuracy offline evaluation and low-latency online inference, with 97.00% measured accuracy on the 200-record streaming simulation.
- The free-tier Colab + Spark environment is good for proof-of-concept work, but has non-trivial constraints: sessions expire, there is only a limited number of parallel jobs, and persistent services like Kafka are not available, which limits the production-readiness of the pipeline.

There are several directions for future work that should be investigated:

- Replacement of the file-based stream simulation with integration of a persistent Kafka cluster (e.g., Confluent Cloud free tier).
- Full deployment of CAMEL Tools morphological analysis to activate the currently zero valued morphological features (f43, f46, f64, f67, f76, f106).
- Investigating Arabic-specific transformer models (such as AraBERT, AraGPT2) to generate sequence-level embeddings as extra feature inputs.
- Evaluation of cross-dataset generalization: train on by_polishing and test on from_title to test robustness to generation strategy shift.
- Use of HDFS or cloud object storage (S3, GCS) as a real distributed file system to replace Google Drive and allow horizontal scaling.

References

- [1] Gehrmann, S., Strobel, H., and Rush, A. M. (2019). GLTR: Statistical detection and visualization of generated text. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 111–116.
- [2] Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., and Goldstein, T. (2023). A watermark for large language models. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023).
- [3] KFUPM-JRCAL. (2024). arabic-generated-abstracts dataset. Hugging Face. <https://huggingface.co/datasets/KFUPM-JRCAL/arabic-generated-abstracts>
- [4] Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Media. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>
- [5] Marz, N. and Warren, J. (2015). Big Data: Principles and Best Practices of Scalable Real-Time Data Systems. Manning Publications.
- [6] Oberländer, L. A. H. and Opitz, J. (2022). Is a prompt and a few samples all you need? Using GPT-4 and Claude for few-shot learning in NLP. arXiv preprint arXiv:2210.15373.
- [7] Solaiman, I., Brundage, M., Clark, J., Askill, A., Herbert-Voss, A., Wu, J., Radford, A., Krueger, G., Kim, J. W., Kreps, S., McCain, K., Newhouse, A., Blazakis, J., McGuffie, K., and Wang, J. (2019). Release strategies and the social impacts of language models. arXiv preprint arXiv:1908.09203.
- [8] Stamatatos, E. (2009). A survey of modern authorship attribution methods. Journal of the American Society for Information Science and Technology, 60(3), 538–556.
- [9] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56–65. doi.org
- [10] Zampieri, M., Malmasi, S., Ljubešić, N., Nakov, P., Ali, A., Tiedemann, J., Scherrer, Yves., & Aepli, N. (2017). Findings of the VarDial evaluation campaign 2017. In P. Nakov, M. Zampieri, N. Ljubešić, J. Tiedemann, S. Malmasi, & A. Ali (Eds.), Proceedings of the Fourth Workshop on NLP for Similar Languages, Varieties and Dialects (pp. 1–15). Association for Computational Linguistics.